

# **Implementação de um sistema de controle de elevador de 5 andares em FPGA utilizando a linguagem de descrição de hardware verilog**

---

**Autores:** Arthur Kolankiewicz, Eduardo Robetti Bedin, Maurício Albiero

**Afiliação:** Universidade Federal da Fronteira Sul (UFFS)



## **Resumo**

Este artigo descreve o desenvolvimento e a implementação de um sistema de controle de elevador de cinco andares em um Field-Programmable Gate Array (FPGA), utilizando a linguagem de descrição de hardware Verilog. O projeto, concebido no contexto da disciplina de Sistemas Digitais da Universidade Federal da Fronteira Sul (UFFS), foca na robustez técnica e na simulação de funcionalidades complexas de um elevador real. As funcionalidades implementadas incluem gerenciamento de chamadas, controle de capacidade de passageiros, modos de operação especiais (emergência e standby) e um mecanismo de bloqueio de andar. A arquitetura modular baseada em uma Finite State Machine (FSM) é detalhada, juntamente com a lógica de hardware que permite a interação com periféricos de uma placa de desenvolvimento, como chaves (*switches*), botões (*keys*) e *displays* de 7 segmentos. O trabalho demonstra a aplicação prática de conceitos de hardware digital e programação para a solução de problemas de controle em tempo real.

**Palavras-chave:** *FPGA, Verilog, Sistemas Digitais, Controle de Elevador, Finite State Machine.*

# 1. Introdução

O controle de sistemas eletromecânicos, como elevadores, é um desafio clássico na engenharia de sistemas digitais. A exigência de operação em tempo real e a complexidade da lógica de sequenciamento tornam a implementação em hardware reconfigurável, como os FPGAs, uma solução tecnicamente apropriada. Este trabalho apresenta o projeto de um sistema de controle de elevador de cinco andares, desenvolvido no âmbito da disciplina de Sistemas Digitais da UFFS.

O objetivo principal do projeto foi a criação de um modelo funcional que não se limitasse ao gerenciamento básico do movimento entre andares, mas que também incorporasse funcionalidades avançadas, essenciais em sistemas de controle reais:

- **Gerenciamento de chamadas:** Atendimento sequencial de requisições internas e externas, com priorização de movimento.
- **Controle de capacidade:** Monitoramento e limitação do número de passageiros para simular a prevenção de sobrecarga.
- **Modo de emergência:** Ativação manual para forçar a descida imediata ao térreo (andar 1).
- **Modo standby:** Mecanismo de economia de energia ativado após um período de inatividade.
- **Bloqueio de andar:** Capacidade de desativar o serviço em um andar específico.

A implementação foi realizada integralmente em Verilog, com foco na descrição do hardware, visando a síntese eficiente e o *deployment* em um dispositivo FPGA.

## 2. Metodologia

A metodologia de desenvolvimento seguiu o fluxo padrão de projeto de hardware digital, partindo da especificação, passando pela descrição em Verilog e culminando na simulação e síntese. O controle sequencial do elevador foi gerenciado por uma Finite State Machine (FSM) síncrona, que é o *core* do sistema.

### 2.1. Criação e Particionamento da Finite State Machine (FSM)

Para este projeto, optou-se por uma FSM única de alto nível para gerenciar os estados de movimento do elevador, devido à sua simplicidade e ao número limitado de andares (5). Os estados definidos são:

- **IDLE (2'b00):** Elevador parado, aguardando chamadas.
- **UP (2'b01):** Elevador em movimento ascendente.
- **DOWN (2'b10):** Elevador em movimento descendente.

O particionamento funcional da lógica de controle é realizado pela separação das tarefas em blocos síncronos e combinacionais distintos, em vez de múltiplas FSMs

interconectadas. A FSM de movimento (*state*) é a máquina principal, e as demais funcionalidades são implementadas como módulos de lógica combinacional e registradores síncronos que a FSM utiliza como entradas e saídas:

- **Módulo de Gerenciamento de Chamadas:** Lógica combinacional que gera os sinais *has\_call\_above* e *has\_call\_below* (entradas da FSM).
- **Módulo de Temporização:** Registrador síncrono (*inactivity\_counter*) que gera o sinal *standby\_mode* (entrada da FSM).
- **Módulo de Controle de Saída:** Lógica combinacional que decodifica o estado atual (*state*) e o andar (*floor*) para os displays de 7 segmentos (saídas).

A transição de estado da FSM principal é determinada pela presença de chamadas pendentes (*calls*), pela direção atual do elevador e pela lógica de priorização de movimento.

## 2.2. Lógica de transição de estados e próximo andar

A lógica de movimento do elevador é implementada para atender às chamadas de forma eficiente, priorizando a direção atual. A transição de estado e a determinação do próximo andar ocorrem em um bloco síncrono, ativado pelo pulso de *clock* e condicionado ao *move\_tick* (pulso de movimento simulado).

O trecho de código Verilog a seguir ilustra a lógica de decisão de movimento, que verifica a existência de chamadas acima (*has\_call\_above*) ou abaixo (*has\_call\_below*) do andar atual, respeitando a capacidade máxima (*is\_full*) e o modo de *standby*:

```
// Move automaticamente quando move_tick ocorrer (apenas se não estiver em standby)
else if (move_tick && !standby_mode) begin
    // Modo normal: atende chamadas (move um andar por vez)
    if (!is_full && calls != 5'b0) begin
        // Decide movimento: sempre um andar por vez
        if (has_call_above && (state == UP || state == IDLE || !has_call_below)) begin
            // Sobe um andar
            floor <= floor + 1'b1;
            state <= UP;
        end else if (has_call_below) begin
            // Desce um andar
            floor <= floor - 1'b1;
            state <= DOWN;
        end else begin
            state <= IDLE;
        end

        // Limpa chamada do andar atual (se houver)
        case (floor)
            3'd1: if (calls[0] && unlocked_floors[0]) calls[0] <= 1'b0;
            3'd2: if (calls[1] && unlocked_floors[1]) calls[1] <= 1'b0;
            3'd3: if (calls[2] && unlocked_floors[2]) calls[2] <= 1'b0;
            3'd4: if (calls[3] && unlocked_floors[3]) calls[3] <= 1'b0;
            3'd5: if (calls[4] && unlocked_floors[4]) calls[4] <= 1'b0;
        endcase
    end else begin
        state <= IDLE;
    end
end
```

A lógica de prioridade é implementada pela condição *has\_call\_above && (state == UP || state == IDLE || !has\_call\_below)*, que garante que o elevador continue subindo se já estiver subindo, ou comece a subir se estiver parado e houver chamadas acima. Apenas se não houver chamadas acima ou se a direção atual for *DOWN* e houver chamadas abaixo, o elevador desce.

## 2.3 Mapeamento para HDL (Verilog)

O mapeamento da FSM para a Linguagem de Descrição de Hardware (HDL) Verilog foi realizado através de blocos *always @(posedge CLOCK\_50)* para a lógica sequencial (transição de estados e atualização de registradores) e blocos *always @(\*)* para a lógica combinacional (saídas e decodificação de *displays*). A utilização de *localparam* (variável do código) para os estados (*IDLE*, *UP*, *DOWN*) e a definição de registradores para o estado atual (*state*) e o andar atual (*floor*) garantem a clareza e a rastreabilidade do *design*.

## 3. Descrição do Sistema

O sistema de controle do elevador é encapsulado no módulo denominado *elevador\_pequeno*.

### 3.1. Interface de Hardware

A interface do módulo com o ambiente externo é definida pelos seguintes sinais, mapeados para periféricos de uma placa de desenvolvimento FPGA:

| Sinal            | Direção | Largura (bits) | Função                                                               |
|------------------|---------|----------------|----------------------------------------------------------------------|
| <b>CLOCK_50</b>  | Entrada | 1              | <i>Clock</i> principal (50 MHz)                                      |
| <b>KEY[3:0]</b>  | Entrada | 4              | Botões de controle (KEY[0] para Emergência)                          |
| <b>SW[9:0]</b>   | Entrada | 10             | Chaves de entrada (Chamadas, Controle de Pessoas, Bloqueio de Andar) |
| <b>HEX[3:0]</b>  | Saída   | 7              | Displays de 7 segmentos para feedback visual                         |
| <b>LEDR[9:0]</b> | Saída   | 10             | LEDs de indicação (Chamadas e Pessoas)                               |
| <b>LEDG[7:0]</b> | Saída   | 8              | LEDs de indicação de estado (Direção, Cheio, Emergência)             |

### 3.2. Lógica de Controle e Funcionalidades

O sistema implementa as seguintes funcionalidades:

- **Controle de Movimento:** Um divisor de frequência gera um pulso de movimento (*move\_tick*) a cada 1 Hz, simulando o tempo de deslocamento entre andares.
- **Gerenciamento de Chamadas:** As requisições de parada são registradas no vetor *calls[4:0]* (chaves SW[4:0]). A lógica de prioridade de movimento atende às chamadas na direção atual.
- **Controle de Capacidade:** O registro *people[1:0]* monitora a capacidade (máximo de 3 pessoas). O elevador não atende chamadas se estiver com a capacidade máxima atingida.
- **Lógica de Display:** Os displays de 7 segmentos (HEX[3:0]) exibem o andar atual, a direção (S/D/P) e o número de pessoas.

## 4. Descrição da Nova Funcionalidade Incluída

O projeto original foi estendido com a inclusão de duas funcionalidades avançadas que simulam cenários de uso reais e demonstram a complexidade do controle implementado: o Modo *Standby* e o Bloqueio de Andar.

### 4.1. Modo Standby (Economia de Energia)

O Modo *Standby* é um recurso de economia de energia. Um contador de inatividade (*inactivity\_counter*) é utilizado para detectar a ausência de atividade do usuário (mudança em SW ou KEY) por um período de 3 segundos (*INACTIVITY\_TIMEOUT*). Se o elevador estiver em IDLE e não em emergência, o modo standby é ativado. Neste modo, o elevador se move automaticamente para o último andar (andar 5) e exibe um indicador visual de baixo consumo nos displays de 7 segmentos.

A implementação do modo *standby* começa com a declaração dos registradores e constantes de tempo:

```
26
27 // Controle de standby
28 reg [9:0] sw_prev_all = 10'b0; // Todos os switches anteriores
29 reg [3:0] key_prev_all = 4'b0; // Todos os botões anteriores
30 reg [27:0] inactivity_counter = 0; // Contador de inatividade (5 segundos = 250M ciclos)
31 reg standby_mode = 1'b0; // Modo standby
32 localparam INACTIVITY_TIMEOUT = 28'd150000000; // 3 segundos a 50MHz
33
```

A detecção de inatividade é crucial para a ativação do modo *standby*. O sistema monitora continuamente a mudança nos valores dos *switches* (SW) e botões (KEY) através da *wire activity*. O bloco síncrono a seguir demonstra a lógica de contagem e ativação: o contador é zerado a cada atividade e, se o elevador estiver em *IDLE* e não em emergência, o contador é incrementado até que o *INACTIVITY\_TIMEOUT* seja atingido, ativando o *standby\_mode*.

```

// Detecta qualquer mudança nos switches ou botões (fora do bloco always)
wire activity = (SW != sw_prev_all) || (KEY != key_prev_all);

always @(posedge CLOCK_50) begin
    if (activity) begin
        // Reset do contador de inatividade quando há atividade
        inactivity_counter <= 0;
        standby_mode <= 1'b0; // Sai do modo standby
    end else if (state == IDLE && !emergency_mode) begin
        // Só conta inatividade quando o elevador estiver parado (IDLE) e não em emergência
        if (inactivity_counter < INACTIVITY_TIMEOUT) begin
            inactivity_counter <= inactivity_counter + 1'b1;
        end else begin
            // Entrou em modo standby após 3 segundos parado sem atividade
            standby_mode <= 1'b1;
        end
    end else begin
        // Se estiver em movimento, reseta o contador
        inactivity_counter <= 0;
    end
end
end

```

## 4.2. Bloqueio de Andar (Controle de Acesso)

A funcionalidade de Bloqueio de Andar permite desativar o serviço em um andar específico (de 1 a 5). O registro *unlocked\_floors*[4:0] é configurado pelas chaves SW[7:5]. Se um andar estiver bloqueado, as chamadas para ele são ignoradas e, se já estiverem pendentes, são limpas, garantindo que o elevador não pare no andar restrito. A lógica de registro de chamadas é *calls* <= *calls* | (*sw\_calls* & *unlocked\_floors*). Além disso, abaixo se encontra uma imagem de onde é possível perceber como funciona a funcionalidade em questão.

```

always @(posedge CLOCK_50) begin
    // Atualiza quais andares estão desbloqueados baseado nos switches SW[7:5]
    // SW[7:5] = 000 (0): todos desbloqueados
    // SW[7:5] = 001 (1): bloqueia andar 1
    // SW[7:5] = 010 (2): bloqueia andar 2
    // SW[7:5] = 011 (3): bloqueia andar 3
    // SW[7:5] = 100 (4): bloqueia andar 4
    // SW[7:5] = 101 (5): bloqueia andar 5
    // SW[7:5] = 110-111 (6-7): todos desbloqueados (valores inválidos, limitado a 5)
    if (blocked_floor_sw >= 3'd1 && blocked_floor_sw <= 3'd5) begin
        // Bloqueia o andar especificado, desbloqueia os demais
        case (blocked_floor_sw)
            3'd1: begin
                unlocked_floors <= 5'b11110; // Bloqueia andar 1
                calls[0] <= 1'b0; // Limpa chamada do andar bloqueado
            end
            3'd2: begin
                unlocked_floors <= 5'b11101; // Bloqueia andar 2
                calls[1] <= 1'b0; // Limpa chamada do andar bloqueado
            end
            3'd3: begin
                unlocked_floors <= 5'b11011; // Bloqueia andar 3
                calls[2] <= 1'b0; // Limpa chamada do andar bloqueado
            end
            3'd4: begin
                unlocked_floors <= 5'b10111; // Bloqueia andar 4
                calls[3] <= 1'b0; // Limpa chamada do andar bloqueado
            end
            3'd5: begin
                unlocked_floors <= 5'b01111; // Bloqueia andar 5
                calls[4] <= 1'b0; // Limpa chamada do andar bloqueado
            end
            default: unlocked_floors <= 5'b11111; // Todos desbloqueados
        endcase
    end else begin
        // Se o valor for 0, 6 ou 7, todos os andares ficam desbloqueados
        unlocked_floors <= 5'b11111;
    end
end
end

```

## 5. Resultados

A implementação em Verilog resultou em um sistema de controle de elevador funcional e robusto, com gerenciamento eficiente de requisições de movimento e estados operacionais. A utilização da FSM demonstrou ser eficaz para a separação da lógica de controle, enquanto o divisor de *clock* simulou o tempo de movimento de forma determinística.

### 5.1. Avaliação da Funcionalidade

A inclusão dos modos de Emergência e *Standby* e do Bloqueio de Andar demonstra a capacidade do projeto de lidar com cenários de uso cruciais em sistemas de controle reais.

- **Modo de Emergência:** Ativado por KEY[0], o sistema limpa todas as chamadas e força a descida imediata ao térreo (andar 1), exibindo o alerta "190" nos displays.
- **Bloqueio de Andar:** A lógica de calls  $\leq$  calls | (sw\_calls & unlocked\_floors) garante que o elevador ignore e limpe chamadas para o andar bloqueado, validando o controle de acesso.

O sistema de feedback visual através dos LEDs e displays de 7 segmentos provou ser uma ferramenta essencial para a depuração e para a interação com o usuário, fornecendo informações claras sobre o andar atual, direção, número de pessoas e estado de emergência/standby.

## 6. Referências Bibliográficas

[1] FERREIRA, Nickerson. Template SBC Artigo TCC (Word). Disponível em: <<https://docentes.ifrn.edu.br/nickersonferreira/disciplinas/pratica-profissional/template-sbc-artigo-tcc-word/view>>.

## Agradecimentos

O desenvolvimento deste artigo contou com o auxílio de ferramentas de Inteligência Artificial, conforme detalhado a seguir:

- **MANUS AI:** Auxílio no amadurecimento de ideias e na estruturação e documentação inicial do artigo.
- **Chat GPT:** Utilizado para esclarecimento de dúvidas pontuais, tanto em relação ao código Verilog quanto a conceitos teóricos de Sistemas Digitais.
- **Cursor:** Utilizado como ferramenta de auxílio na análise, depuração e otimização do código Verilog, complementando o entendimento da implementação e acelerando o processo de desenvolvimento.

A autoria, a compreensão e a responsabilidade final por todos os artefatos entregues são exclusivas dos autores do artigo.

---

## **NOTAS DE RODAPÉ:**

- link do Artigo + Código Verilog no GitHub ->  
<https://github.com/akolankiewicz/T3-SistemasDigitais2025.2>